



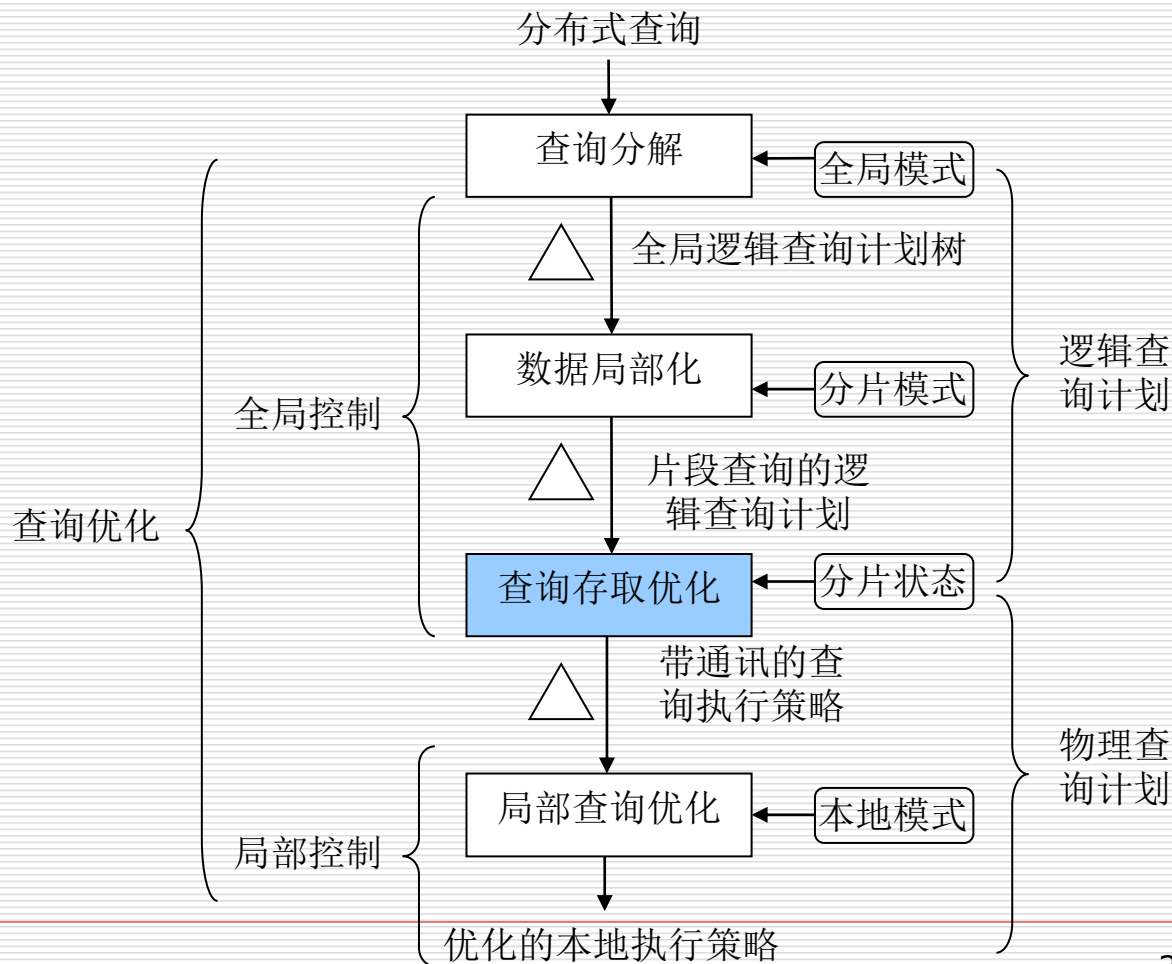
分布与并行数据库

5 分布式查询的存取优化

主讲教师：鄂金龙
中国人民大学信息学院



分布式查询处理概述



分布式查询处理概述

□ 查询存取优化

- 输入：片段查询
- 找到好的（不必最优的）全局调度
 - ✓ 最小代价函数
 - ✓ 分布Join处理
 - 稠密树与线性树
 - 需要传输的关系
 - 全部传输与按需传输
 - ✓ 确定使用半连接
 - 使用半连接可减少通信代价
 - ✓ Join方法
 - 嵌套循环与顺序join（合并join与hash join）

分布式查询处理概述

□ 基于代价的优化

■ 搜索空间(Solution space)

- ✓ 等价的代数表达式 (query trees)

■ 代价函数(Cost function)

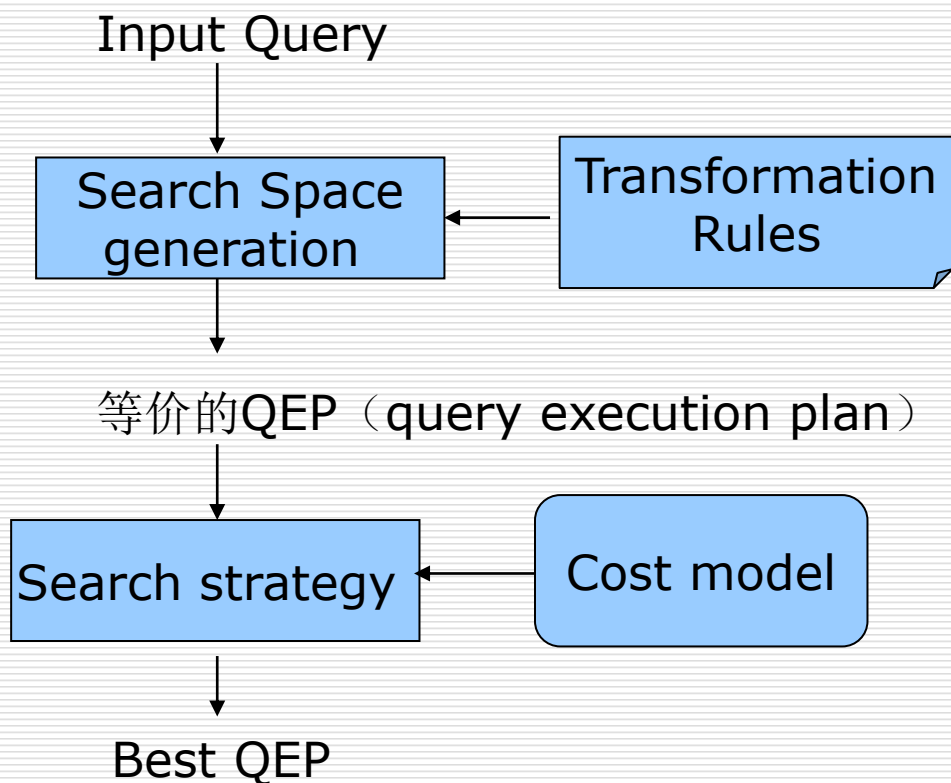
- ✓ I/O 代价 + CPU 代价 + communication代价
- ✓ 不同的分布环境中各因素权重不同 (LAN vs WAN)
- ✓ 最大吞吐量

■ 搜索算法(Search algorithm)

- ✓ 如何在解决空间中移动
- ✓ 穷举搜索, 启发式算法 (递归改进、模拟退火、遗传法等)

分布式查询处理概述

□ 查询优化过程



- 查询优化
- ✓ 最小化查询代价

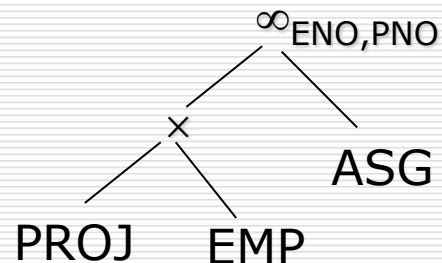
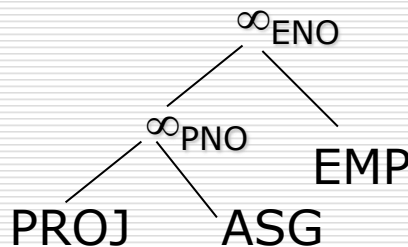
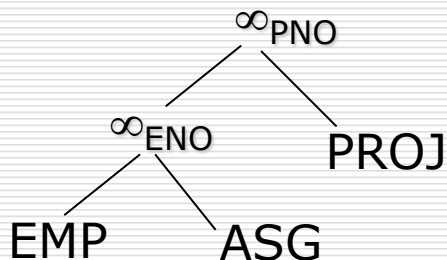
分布式查询处理概述

□ 搜索空间(Search Space)

- 搜索空间由可选择的执行计划描述
- 侧重Join树
- 若存在n个关系，有 $O(N!)$ 等价的Join树

例如，

```
select ENAME,RESP
From EMP,ASG,PROJ
Where EMP.ENO=ASG.ENO
And ASG.PNO=PROJ.PNO
```



分布式查询处理概述

□ 搜索空间(Search Space)

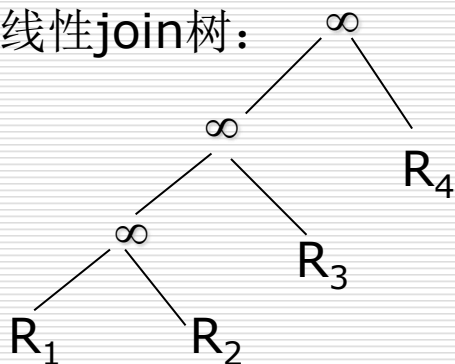
- 由启发式规则约束

如一元操作先于二元操作执行

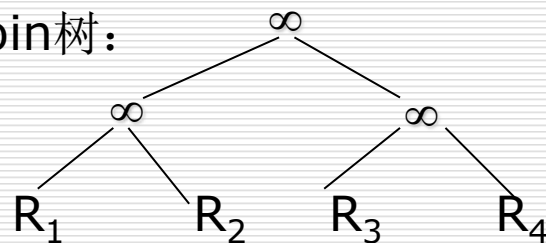
- 约束join树的形状

只考虑线性树而忽略稠密树

线性join树:



稠密join树:



分布式查询处理概述

- ✓构造所有可能的执行计划;
- ✓尽早剪裁不是优化的计划;
- ✓Is also exhaustive;
- ✓关系数少时, 优化代价可接受;

□ 搜索策略(Search Strategy)

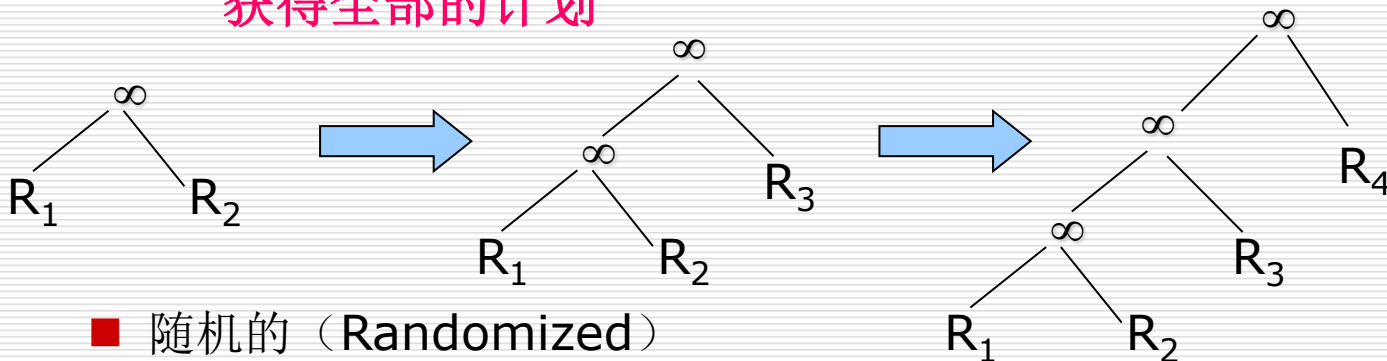
- 如何在搜索空间中构建查询计划
- 确定的 (Deterministic)
 - ✓ 开始于基关系, 每步加一个关系构建执行计划
 - ✓ 动态规划法: 宽度优先 (breadth-first)
 - ✓ 贪婪法: 深度优先 (depth-first) (只构造一个计划)
- 随机的 (Randomized)
 - ✓ 从一个特定的点起始搜索最优执行计划 (首先应用贪婪法构建一个或多个执行计划, 接着通过邻居关系替换来改进执行计划)
 - ✓ 综合考虑优化时间和执行时间
 - ✓ 适合大于 5-6 个关系
 - ✓ 模拟退火 (Simulated annealing)
 - ✓ 递归改进 (Iterative improvement)

分布式查询处理概述

□ 搜索策略(Search Strategy)

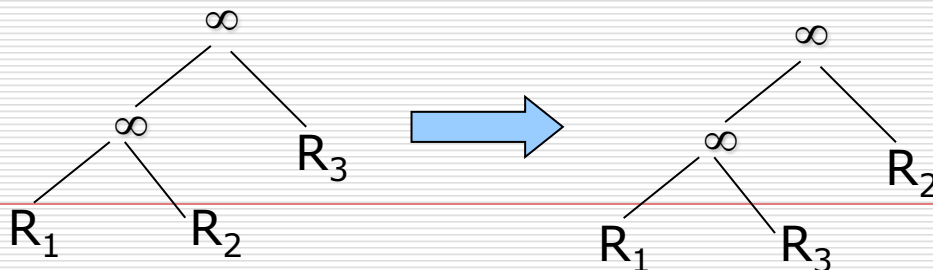
■ 确定的 (Deterministic)

✓ 从基关系开始，每步连接一个或多个关系，直到获得全部的计划



■ 随机的 (Randomized)

✓ 随机交换操作关系





分布查询的存取优化

- 基本概念
- 优化的理论基础
- 半联接优化方法
- 枚举法优化技术
- 集中式系统中的查询优化算法
- 分布式查询优化算法
- 大数据数据库的查询处理与优化

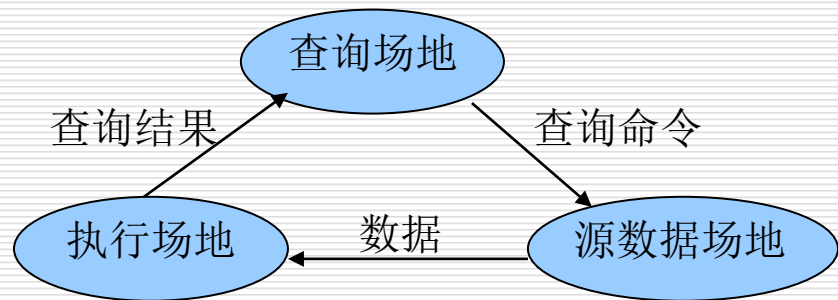
基本概念

□ **分布执行过程**实际上就是从查询场地发出查询命令、从数据源获取数据、确定最佳的执行场地和返回执行结果的过程。

■ **查询场地**：指发出查询命令和存储最终查询结果的场地。查询场地也称最终结果文件。

■ **源数据场地**：指查询命令需要访问的数据副本所在的场地，可能涉及到一个或一个以上的场地。源数据场地也称源数据文件。

■ **执行场地**：指查询操作执行所在的场地。执行场地可以和查询场地或源数据场地处于同一场地，也可不处于同一场地。执行场地也称中间结果文件。



基本概念

□ 分布执行策略举例

■ 有关系EMP和DEPT。

✓ **EMP{ENO, ENAME, BIRTH, SALARY, DNO}**
(**ENO**为主键)

ENO, ENAME, BIRTH, SALARY, DNO 分别为雇员编号 雇员姓名 出生日期 工资 部门号

✓ **DEPT{DNO, DNAME}**
(**ENO**为主键)

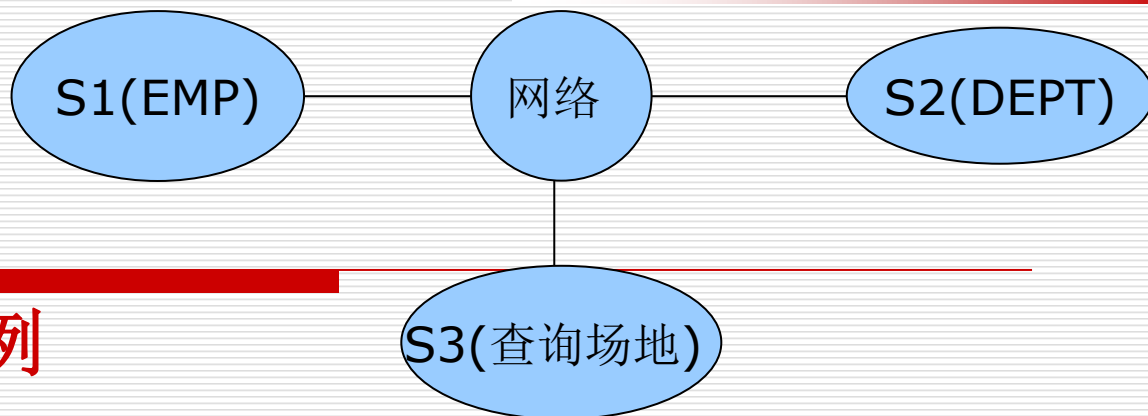
DNO, DNAME 分别为部门号, 部门名称

■ 假设

(1) **EMP**: 元组数: **10000**, 元组大小: **100B**, 关系大小:
 $100 \times 10000 = 1000\text{KB}$

(2) **DEPT**: 元组数: **100**, 元组大小: **35B**, 关系大小:
 $35 \times 100 = 3.5\text{KB}$

基本概念



□分布执行策略举例

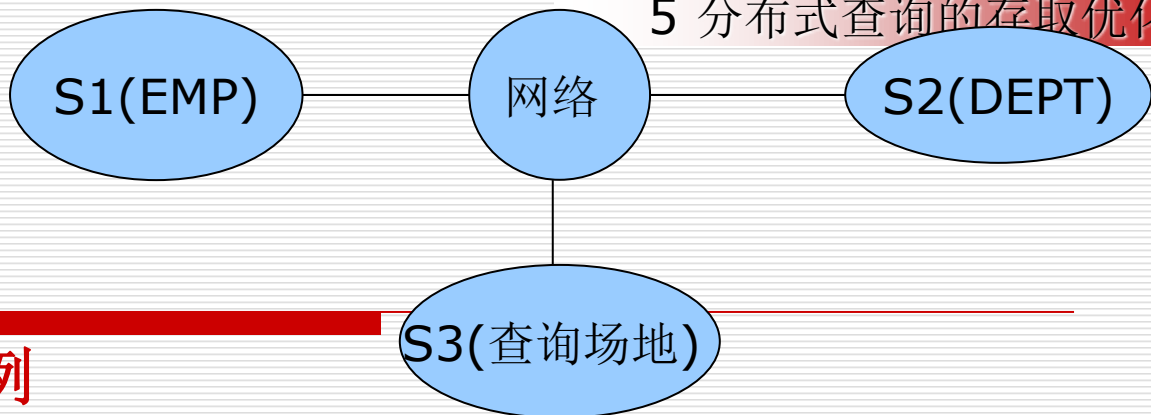
(3) 存在三个场地，S1、S2和S3。如图：

查询要求： 查询每个雇员的姓名及所在单位。

则：（1）**SQL语句：** **SELECT ENAME, DNAME**
FROM EMP, DEPT
WHERE EMP.DNO=DEPT.DNO

（2）关系表达式： $\pi_{ENAME,DNAME} (EMP \bowtie DEPT)$

假设： 结果元组大小40字节，S3为查询场地
结果关系大小： $40 \times 10000 = 400KB$



基本概念

□ 分布执行策略举例

■ 策略（设结果为**R**，以传输代价为主）

策略1:

S3为执行场地，则需传输**EMP**、**DEPT**

传输量= $1000K + 3.5K = 1003.5K$

策略2:

S2为执行场地，则需传输**EMP**到**S2**，结果**R**传输到**S3**。

传输量= $1000K + 400K = 1400K$

策略3:

S1为执行场地，则需传输**DEPT**到**S1**，结果**R**传输到**S3**。

传输量= $3.5K + 400K = 403.5K$

- 从上面三个策略看，选择不同的执行场地，传输代价差别很大。应选择最低的传输代价。但组成系统的环境不同，优化的侧重点也不同。

基本概念

□分布查询的存取优化的目标

- 远程网：主要考虑通信开销，使通信代价最小。
- 局域网：需同时考虑通信代价和本地处理代价，使综合代价最小。

□优化的内容

优化是在片段查询的基础上进行的实际物理副本查询操作的优化。具体如下：

输入：片段查询表达式

输出：分布执行计划

基本概念

□ 查询存取优化 内容：

- **确定片段查询需访问的物理副本。**通常：①本场地上的物理副本优先；②若二元运算存在尽量选择本场地上的二元运算；③数据最小的物理关系应被优先选中；④网络通信代价小的应优先选中。
- **确定片段查询表达式操作执行的最优顺序。**包括从叶到根的执行和同一层叶子上表达式执行的先后，特别是对查询树上的并操作和联接操作的执行次序的确定，其代价差别很大。
- **选择执行每个操作的方法。**如：尽量将同一场地上的、同一物理副本的全部操作组合在一起统一考虑完成。

存取优化的理论基础

□代价函数(the total time or the response time)

■总的时间 (Total Time) -所有时间组件的和

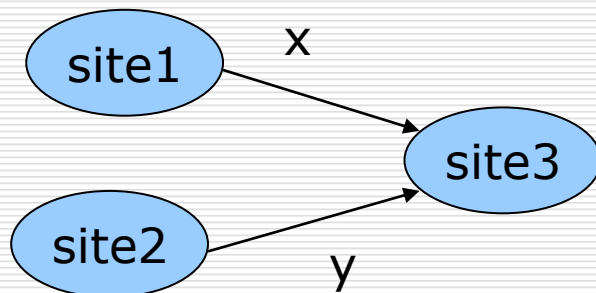
- ✓ 减少每一个组件的时间代价
- ✓ 尽可能减少每个代价组件的代价
- ✓ 优化资源利用率，增加系统吞吐率

■ 响应时间 (Response Time) -从查询开始到执行结束所用的时间

- ✓ 尽可能并行执行 (Do as many things as possible in parallel)
- ✓ 增加总的活动 (activity) 可能会增加总时间

存取优化的理论基础

□ 例子:



■ 假设只考虑通信代价

✓ 总时间 = $2 \times \text{消息启动时间} + \text{单位传输代价} \times (x + y)$

✓ 响应时间 = $\max(\text{从1传输}x\text{到3的时间}, \text{从2传输}y\text{到3的时间})$

存取优化的理论基础

□ 代价模型

主要指传输代价 (C_{com})、I/O代价 (C_{IO}) 和CPU代价 (C_{cpu})

$$\text{Total cost} = C_{com} + C_{IO} + C_{cpu}$$

■ 传输代价

费用和延迟。其中，费用起决定作用。

传输费用是指通信中的整个传输开销，即传输的数据量。

模型为： $C_{COM}(X) = C_0 + C_1 * X$

其中： C_0 ：场地间传输数据的启动所需的固定费用（启动一次），简称启动代价；

C_1 ：网络单位传输数据费用，简称单位传输代价；

X ：需传输的数据量。

存取优化的理论基础

■ I/O代价

模型为： $C_{IO}(X) = [X/P] * C_{IO}$

其中： P ：页面的大小； C_{IO} ：为每页平均访问代价；
 X ：数据量大小。

■ CPU代价

模型： $C_{CPU}(X) = X * C_{CPU}$

其中： C_{CPU} ：单位指令代价； X ：为指令数。

通常具有下面的统计值：

广域网环境： $C_{COM} / C_{IO} = 20:1$ ；

局域网环境： $C_{COM} / C_{IO} = 1.6:1$ 。

可见，在广域网环境，以传输代价为主；在局域网环境，需综合考虑传输代价和局部代价。

存取优化的理论基础

□ 查询模型

主要代价因素：中间关系大小

■ 数据库特征参数

假设 R 为一关系

- ✓ **关系的基**：指关系 R 包含的元组个数，记为**Card(R)**
- ✓ **属性的长度**：指属性 A 定义的取值字节数，记为**Length(A)**
- ✓ **元组的长度**：关系 R 中每个元组的字节数，记为**Length(R)**，
Length(R) = $\sum \text{Length}(A_i)$
- ✓ **关系的大小**：关系 R 所包含的字节数，记为**Size(R)**
Size(R) = Card(R) * Length(R)
- ✓ **属性的特征值**：指关系 R 中属性 A 不同的属性值个数，记为**Val(A)**
- ✓ **属性 A 的值域**：记为**Dom(A)**
- ✓ **属性 A 的最大值和最小值**：记为**Max(A)**和**Min(A)**

存取优化的理论基础

■ 关系运算的特征参数

假设：R、S为关系。

✓ 选择运算 ($S = \sigma_F(R)$)

选择度：

满足选择谓词F的元组与R元组总数之比，记为 ρ

基数： $\text{Card}(S) = \rho * \text{Card}(R)$

关系的宽度：

$\text{Length}(S) = \text{Length}(R)$

➔ $\text{Length}(S.A) = \text{Length}(R.A)$

存取优化的理论基础

✓ 选择运算(中间关系大小)

选择度 ρ 的具体计算方法:

①等值比较

$S = \sigma_{A=X}(R)$, 其中 A 是 R 的属性, X 是常数。

则 $\rho = 1 / \text{Val}(R, A)$

②非等值比较

$S = \sigma_{A>X}(R)$ 时: $\rho = (\text{Max}(A) - X) / (\text{Max}(A) - \text{Min}(A))$

$S = \sigma_{A<X}(R)$ 时: $\rho = (X - \text{Min}(A)) / (\text{Max}(A) - \text{Min}(A))$

③不等比较

$S = \sigma_{A \neq X}(R)$ 时: $\rho = (\text{Val}(R, A) - 1) / \text{Val}(R, A)$

④多属性选择条件

$S = \sigma_{C_i \text{ AND } C_j}(R)$ 时: $\rho = \rho_i * \rho_j$

$S = \sigma_{C_i \text{ OR } C_j}(R)$ 时: $\rho = 1 - (1 - \rho_i)(1 - \rho_j) = (\rho_i + \rho_j - \rho_i * \rho_j)$

存取优化的理论基础

✓选择运算 ($S = \sigma_F(R)$)

不同值的个数:

a. 设B不属于选择谓词F, 其值均匀分布。

$$\text{Val}(S, B) = \begin{cases} \text{Card}(S) & \text{当Card}(S) < \text{Val}(R, B)/2 \text{时} \\ (\text{Card}(S) + \text{Val}(R, B))/3 & \text{当Val}(R, B)/2 \leq \text{Card}(S) < 2\text{Val}(R, B) \text{时} \\ \text{Val}(R, B) & \text{当Card}(S) \geq 2\text{Val}(R, B) \text{时} \end{cases}$$

b. 设B属于选择谓词F

若 $B = X$ (值), 则: $\text{Val}(S, B) = 1$

若B与选择谓词F相关且为关键字,

则: $\text{Val}(S, B) = p * \text{Val}(R, B)$

存取优化的理论基础

✓ **选择运算** ($S = \sigma_F(R)$)

✓ **举例**: 设 $\text{Card}(R)=100$, $\text{Val}(R,B)=70$

令 $\rho=0.1$

则: $\text{Card}(S) = \rho * \text{Card}(R) = 0.1 * 100 = 10$

$\because \text{Card}(S)=10$, $\text{Val}(R,B)/2=35$

$\therefore \text{Card}(S) < \text{Val}(R,B)/2$, $\text{Val}(S,B) = \text{Card}(S) = 10$

令 $\rho=0.5$

则: $\text{Card}(S) = \rho * \text{Card}(R) = 0.5 * 100 = 50$

$\because \text{Card}(S)=50$, $\text{Val}(R,B)/2=70/2=35$

$\therefore \text{Val}(R,B)/2 < \text{Card}(S) < 2 * \text{Val}(R,B)$,
 $\text{Val}(S,B) = (\text{Card}(S) + \text{Val}(R,B)) / 3 = 40$

存取优化的理论基础

■ 投影运算 ($S=\pi_A(R)$)

✓ 基数:

①如果投影涉及单个属性A: $\text{Card}(S) = \text{Val}(R, A)$

②如果A中包含关键字: $\text{Card}(S) = \text{Card}(R)$

③投影涉及多个属性, 且有 $\prod_{A_i \in \text{AttrList}} \text{Val}(R, A_i) < \text{Card}(R)$

则 $\text{Card}(S) = \prod_{A_i \in \text{AttrList}} \text{Val}(R, A_i)$

✓ 关系的宽度:

$\text{Length}(S) = \sum \text{Length}(A_i) (A_i \in A)$

$\text{Size}(S) = \text{Card}(S) * \text{Length}(S)$

$\text{Size}(S) < \text{Size}(R)$

✓ 不同值的个数:

$\text{Val}(S, A) = \text{Val}(R, A)$

存取优化的理论基础

- **并、交与差**（如果结果关系执行消除重复操作，则都只能计算出结果关系基数的上限和下限，因此通常采用平均值作为基数的估计值。）

- ✓ **基数：**

- ①并运算

- 如果不消除重复，则结果关系基数等于两个关系基数之和：

- $$\text{Card}(T) = \text{Card}(R) + \text{Card}(S)$$

- 如果消除重复，则结果关系基数最大可大至两个关系基数之和，最小可小至两个关系基数的较大者，因此有：

- $$\text{Max}\{\text{Card}(R), \text{Card}(S)\} < \text{Card}(T) < \text{Card}(R) + \text{Card}(S)$$

- 在估计中使用中间值作为结果关系基数：

- $$(\text{Max}\{\text{Card}(R), \text{Card}(S)\} + \text{Card}(R) + \text{Card}(S)) / 2$$

存取优化的理论基础

■ 并、交与差

✓ 基数：

②交运算

交运算的结果关系最小可以是空，最大可以等于两个关系的较小者，因此按取区间中间值的方法估计结果关系的基数为较小关系基数的一半：

$$\text{Card}(T) = \text{Min}\{\text{Card}(R), \text{Card}(S)\} / 2$$

③差运算

对于两个关系的差运算 **$R-S$** ，其结果关系基数的区间为：

$$\text{Card}(R) - \text{Card}(S) < \text{Card}(T) < \text{Card}(R)$$

其中 **$\text{Card}(R) - \text{Card}(S)$** 在 **$S$** 包含 **$R$** 中所有元组时 **$\text{Card}(R) - \text{Card}(S) = 0$** 。

结果关系基数： **$\text{Card}(T) = (2\text{Card}(R) - \text{Card}(S)) / 2$**

存取优化的理论基础

■笛卡尔积

✓基数:

$$\text{Card}(T) = \text{Card}(R) * \text{Card}(S)$$

✓元组的长度:

$$\text{Length}(T) = \text{Length}(R) + \text{Length}(S)$$

✓属性不同值的数量:

笛卡尔积结果关系中属性不同值的数量等于其原关系中 对应属性的不同取值的数量:

$$\text{Val}(T, A) = \text{Val}(R, A) \text{ 或者}$$

$$\text{Val}(T, A) = \text{Val}(S, A)$$

存取优化的理论基础

■ 连接运算 $S=R \bowtie T$, ($R.a=T.b$)

- ✓ **基数**: 存在 $\text{Card}(S) \leq \text{Card}(R) \times \text{Card}(T)$

具体分下面几种情况:

- ✓ **基本计算形式** (ρ 为连接选择度)

$$\text{Card}(S) = \rho * \text{Card}(R) * \text{Card}(T)$$

- 若 S 以 b 为关键字, a 为外来关键字

$$\text{Card}(S) = \text{Card}(R)$$

- ✓ **关系的宽度**:

$$\text{Length}(S) = \text{Length}(R) + \text{Length}(T)$$

$$\text{Length}(S.a) = \text{Length}(R.a)$$

$$\rho(R, S) = \frac{\text{card}(R \bowtie S)}{\text{card}(R) \bullet \text{card}(S)}$$

通常: $\text{Card}(R \bowtie T) = \text{Card}(R) * \text{Card}(T) / \max(\text{Val}(R, a), \text{Val}(T, a))$

当 $\text{Val}(R, a) \leq \text{Val}(T, a)$, R 中每个元组有 $1/\text{Val}(T, a)$ 机会与 S 中元组连接

存取优化的理论基础

■ 连接运算 $S=R \bowtie T$, ($R.a=T.b$)

✓不同值的个数:

● 设a为连接属性

$$\text{Val}(S, a) \leq \text{Min}(\text{Val}(R, a), \text{Val}(T, b))$$

● 若c不为连接属性

$$\text{Val}(S, c) \leq \text{Val}(R, c) \text{ (c为R的属性)}$$

$$\text{Val}(S, c) \leq \text{Val}(T, c) \text{ (c为T的属性)}$$

存取优化的理论基础

■ 半连接运算 $S=R \bowtie T$, ($R.a=T.b$)

半连接操作是全连接操作的一种缩减。是一种导出操作, 且具有不对称性。如: 半连接操作 $(R \bowtie T)$ 是 R 与 T 自然连接后在 R 上的投影, 描述为: $R \bowtie T = \pi_{Attr(R)}(R \Join T)$

存在: $Card(R \bowtie T) \leq Card(R)$ $R \bowtie T \neq T \bowtie R$

- ✓ 基数: $Card(S) = \rho * Card(R)$ ρ 为半连接选择度
- ✓ 关系的宽度: $Length(S) = Length(R)$
- ✓ 不同值的个数:

a. 设 a 为连接属性

$$Val(S, a) = \rho * Val(R, a)$$

b. 若 c 不为连接属性

$$Val(S, c) \leq Val(R, c)$$

半连接优化方法

- 对**连接操作的优化**有两种趋势：
- 一种为采用**半连接技术**，减少连接操作的操作数，以降低传输费用；
- 另一种为采用**全连接技术**，主要考虑局部代价。
- 一个系统需根据其目标综合确定其优化算法。

半连接优化方法

□ 半连接的作用

假设有雇员关系EMP和部门关系DEPT，

EMP: Card (EMP) = 10000; DEPT: Card (DEPT) = 100

其中，关系EMP保存在场地S1，关系DEPT保存在场地S2，如下图所示。

现有查询要求：在场地S2上查询部门名称和部门经理姓名，选择最优的执行策略，这里只考虑数据传输代价。查询的SQL语句如下：

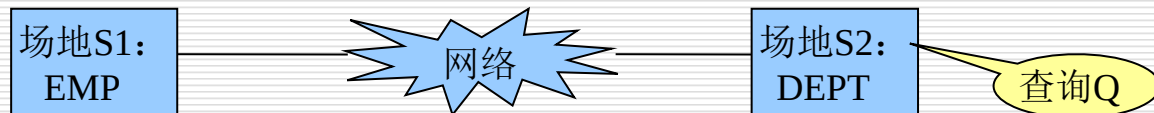
**SELECT DNAME, ENAME
FROM DEPT, EMP
WHERE DEPT.MgrNO = EMP.ENO**

关系代数表达式为：

$Q = \pi_{DNAME, ENAME}(DEPT \bowtie EMP)$

属性	ENO	ENAME	
长度(B)	4	35	

属性	DNO	DNAME	MgrNo	...
长度(B)	4	35	4	



半连接优化方法

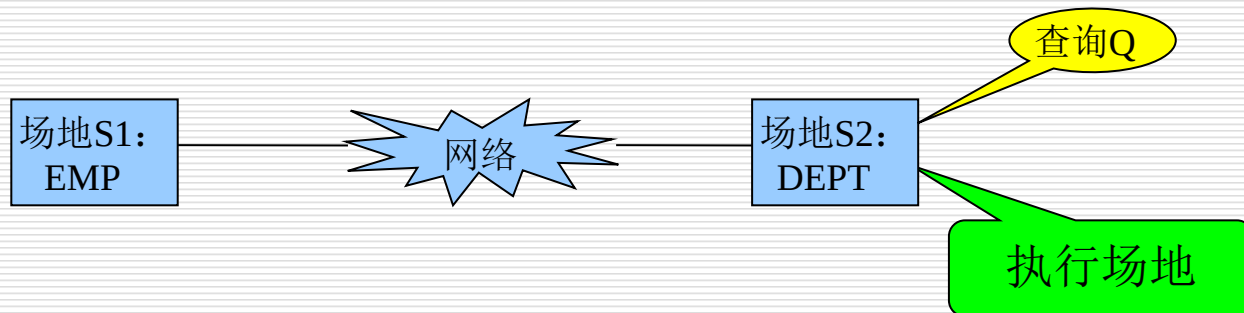
□ 半连接的作用

下面通过三种查询策略分析其代价评估（**COST**）

策略1： 执行场地设在S2

需将EMP的Eno和Ename属性传送到S2场地

$$\begin{aligned}\text{COST} &= (\text{Length}(\text{Eno}) + \text{Length}(\text{Ename})) * \text{Card}(\text{EMP}) \\ &= 39 * 10000 = 39\text{KB}\end{aligned}$$



半连接优化方法

□ 半连接的作用

策略2：执行场地设在S1

需将**DEPT**的**Dname**和**Mgno**属性传送到**S1**场地，操作后，再将结果传回场地**S2**。设**R**为结果。

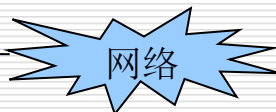
$$\begin{aligned}\text{COST1} &= (\text{Length}(\text{Dname}) + \text{Length}(\text{Mgno})) * \text{Card}(\text{DEPT}) \\ &= 39 * 100 = 3.9\text{KB}\end{aligned}$$

$$\begin{aligned}\text{COST2} &= (\text{Length}(\text{Dname}) + \text{Length}(\text{Ename})) * \text{Card}(\text{R}) \\ &= 70 * 100 = 7\text{KB}\end{aligned}$$

$$\text{COST} = \text{COST1} + \text{COST2} = 10.9\text{KB}$$



场地S1:
EMP



场地S2:
DEPT

查询Q

查询场地

半连接优化方法

□ 半连接的作用

策略3：采用半连接

- ①将DEPT的Mgno属性传送到S1场地，
即将 $D1 = \pi_{Mgno}(DEPT)$ 传送到S1场地。

$$COST1 = Length(Mgno) * Card(DEPT) = 4 * 100 = 0.4KB$$

- ②在场地S1。完成EMP与D1的连接，即实现 $E1 = EMP \bowtie D1$ ，
则： $Card(E1) = 100$ 。

将E1的属性Eno和Ename传到S2，
即将 $E2 = \pi_{Eno, Ename}(E1)$ 传到S2。

$$COST2 = (Length(Eno) + Length(Ename)) * Card(E1) \\ = 39 * 100 = 3.9KB$$

- ③在场地S2上计算： $R = DEPT \bowtie E2$ 。

$$COST = COST1 + COST2 = 0.4 + 3.9 = 4.3 KB。$$

策略3相当于： $EMP \bowtie DEPT = (EMP \bowtie DEPT) \bowtie DEPT$ 。

①②实现 ③实现

半连接优化方法

□ 半连接优化算法

输入信息：位于不同场地上的两个关系R和S

输出信息：实现 $R \bowtie S$ ($R.A=S.B$)

算法：（设S的尺寸小于R）

- （1）在S所在场地上计算 $S' = \pi_B(S)$;
- （2）传送S'到R场地
- （3）在R场地上计算 $R' = R \bowtie S' = R \bowtie S$
- （4）将R'传到S所在场地
- （5）在S所在场地上计算

$$R' \bowtie S = (R \bowtie S) \bowtie S = R \bowtie S$$

半连接优化方法

□ 传输代价的比较

假设：

关系**R**和**S**分别在不同的场地上，**C₀**为启动代价，**C₁**为单位传输代价。

■ 设：在**S**所在的场地上执行，则传输关系**R**实现**R∞S**的代价

$$\begin{aligned} C &= C_0 + C_1 * (\text{Length}(R) * \text{Card}(R)) \\ &= C_0 + C_1 * \text{Size}(R) \end{aligned}$$

半连接优化方法

□ 传输代价的比较

采用半连接的传输代价（需传输**S'**和**R'**）

$$\begin{aligned}C_{\infty} &= C_{S'} + C_{R'} \\&= C_0 + C_1 * (\text{Length}(S') * \text{Card}(S')) + \\&\quad C_0 + C_1 * (\text{Length}(R') * \text{Card}(R')) \\&= 2 C_0 + C_1 * (\text{Length}(S') * \text{Card}(S') + \text{Length}(R') * \\&\quad \text{Card}(R')) \\&= 2 C_0 + C_1 * (\text{Size}(S') + \text{Size}(R'))\end{aligned}$$

分析：如果有： $C_{\infty} \geq C_{\alpha}$

则： $C_0 + C_1 * \text{Size}(R) \geq 2 C_0 + C_1 * (\text{Size}(S') + \text{Size}(R'))$

$$C_0 / C_1 + \text{Size}(S') + \text{Size}(R') \leq \text{Size}(R)$$

枚举法优化技术

- 查询操作的代价评估：综合考虑局部代价和传输代价。
 - 若侧重传输代价，局部代价可以忽略不计时，采用半连接技术较好；
 - 侧重局部代价时，采用直接连接比采用半连接优越。
- 枚举法是基于直接连接的实现方法。

枚举法优化技术

□ 常见的直接连接算法主要有：

- 嵌套循环连接算法(**nest-loop**)
- 归并排序连接算法 (**merge-scan**)
- 哈希连接算法 (**Hash**)
- 基于索引的连接算法

□ 对每种直接连接算法进行代价估计：考虑执行连接操作所需的磁盘**I/O**代价。

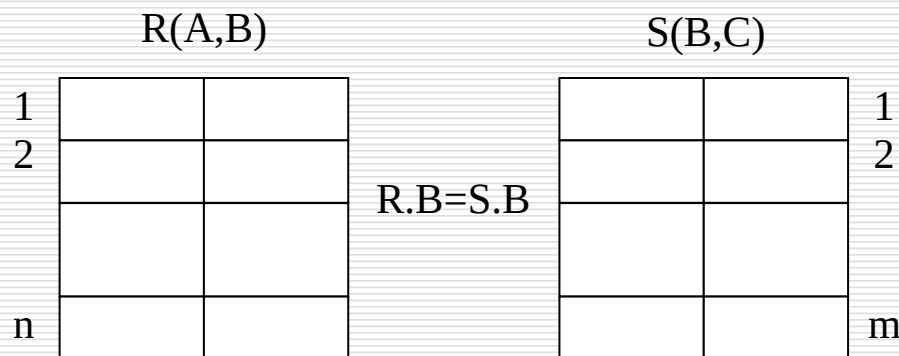
□ 对磁盘**I/O**代价的估计主要依赖于数据库的特征参数。

枚举法优化技术

□ 嵌套循环连接算法

- 是一种最简单的连接算法，原理是对连接操作的两个关系中的一个仅读取其元组一次，而对另一个关系中的元组将多次读取。
- 特点是可以用于任何大小的关系间的连接操作，不必受连接操作所分配的内存空间大小的限制。

假设有关系 $R(A,B)$ 和 $S(B,C)$ ，
 $\text{Card}(R)=n$ ， $\text{Card}(S)=m$ ，
现在要执行两个关系在属性 B 上的连接操作。



枚举法优化技术

□ 基于元组的嵌套循环连接

- 循环以关系中的元组为单位进行操作，具体的执行算法如下：

Result= $\emptyset$ /*初始化结果集合*/

For each tuple s in S

外关系S

For each tuple r in R

内关系R

If r.B=s.B Then /*元组r和元组s满足连接条件*/

Join r and s as tuple t;

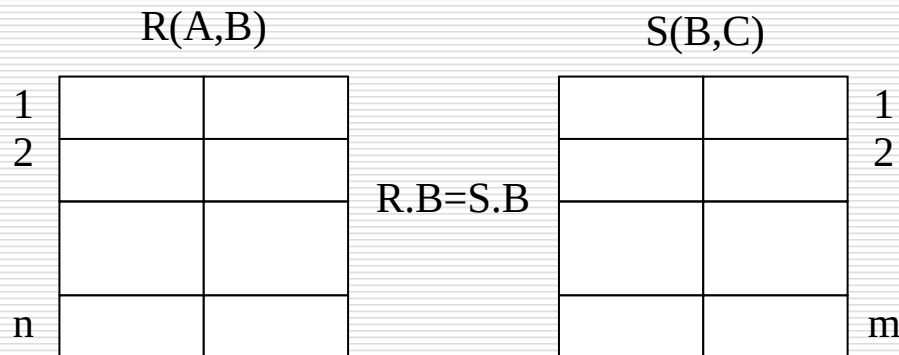
Output t into Result; /*输出连接结果元组*/

End If

End For

End For

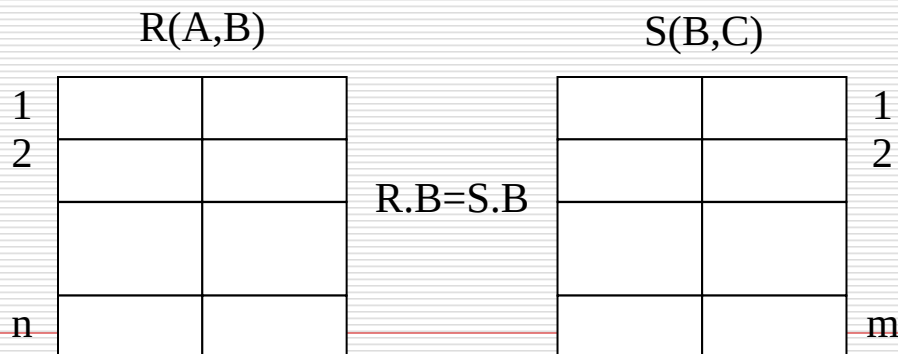
Return Result



枚举法优化技术

□ 基于元组的嵌套循环连接

- 在执行嵌套循环连接时，仅对外关系进行1次读取操作，而对内关系则需要进行多次读取。
- 如果不进行优化的话，执行代价很大，以磁盘IO代价计算最多可能多达 $\text{Card}(S) * \text{Card}(R) = mn$ 次。
- 通常对这种算法进行修改，一种方法是通过尽可能多地使用内存以减少磁盘IO次数；另一种方法是使用连接属性上的索引，以减小参与连接元组的数量。



枚举法优化技术

□ 基于块的嵌套循环连接

- 通过尽可能多的使用**内存**，**减少**读取元组所需的**I/O次数**。
- 对连接操作的两个关系的访问均按块（也称为页面）进行，同时使用尽可能多的内存来存储嵌套循环中外关系的块。

假设：为连接操作分配的内存缓冲区大小为 **M** 个块，有 $\text{Block}(\mathbf{R}) \geq \text{Block}(\mathbf{S}) \geq \mathbf{M}$ ，

- **实现过程**：首先选取**较小的关系**作为**外关系**，本例即关系 **S**。将**1**到**M-1**块分配给关系 **S**，第**M**块分配给关系 **R**。
- 将外关系 **S** 按照 **M-1** 个块的大小分为多个子表，并重复地将这些子表读取到内存缓冲区中，用于重复地依次读取关系 **R** 的每一个块。
- 对于内存缓冲区中元组的连接操作，先在 **M-1** 个块的外关系 **S** 元组的连接属性上构建查找结构，再从内关系 **R** 在内存中的块中取元组，通过查找结构与 **S** 中的元组连接。

枚举法优化技术

基于块的嵌套循环连接

算法:

Result= $\emptyset$ /*初始化结果集合*/

Buffer=**M** /*内存缓冲区*/

For each **M-1** **in** **Block(S)** /*每次从外关系**S**中读取**M-1**个块到内存缓冲区*/

Read **M-1** **of** **Block(S)** **into** **Buffer**;

For each **block** **in** **Block(R)** /*每次从内关系**R**中读取**1**个块到内存缓冲区*/

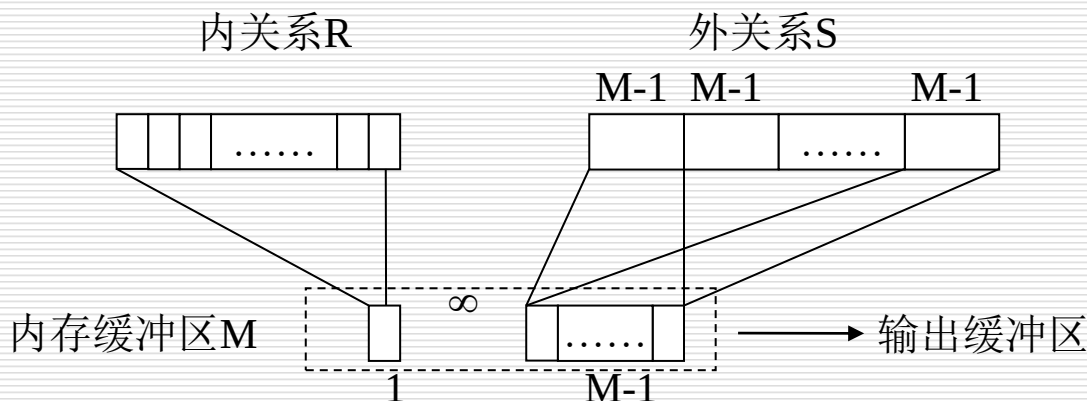
Join **M-1** **of** **Block(S)** **and** **1** **of** **Block(R)** **in** **Buffer**; /*在内存中对元组执行连接*/

Output **t** **into** **Result**;

End For

End For

Return **Result**



枚举法优化技术

□ 基于块的嵌套循环连接方法的代价估计

假设R和S占用的块分别为Block(R)和Block(S)，M为内存缓冲区大小（块数）。一次I/O可读取一个数据块。则共需读取关系S的块数为Block(S)，共需读取R的块数为 $Block(R) * Block(S) / (M - 1)$ 。

$$\text{总} I / O \text{ 次数} = Block(S) + \frac{Block(S)}{M - 1} * Block(R)$$

说明：

如果连接关系 $Block(R) * Block(S)$ 的值远远大于内存缓冲区大小M时，认为连接的代价近似等于 $Block(R) * Block(S)$ 。

这种算法能够适用于任意大小的关系之间的连接执行，因此基于块的嵌套循环连接算法依然广泛应用于现有的数据库系统中。

枚举法优化技术

□ 嵌套循环连接方法举例

- 假设连接的两个关系 R 和 S， $\text{Block}(R)=2000$ ， $\text{Block}(S)=500$ ，内存缓冲区 $M=51$ 。则：

外关系为 S

内存分配： $M-1=50$ 分配给 S，再循环读取关系 R 的 2000 个块。

根据公式：

$$\text{Block}(S) + \frac{\text{Block}(S)}{M-1} * \text{Block}(R)$$

连接的执行代价为：

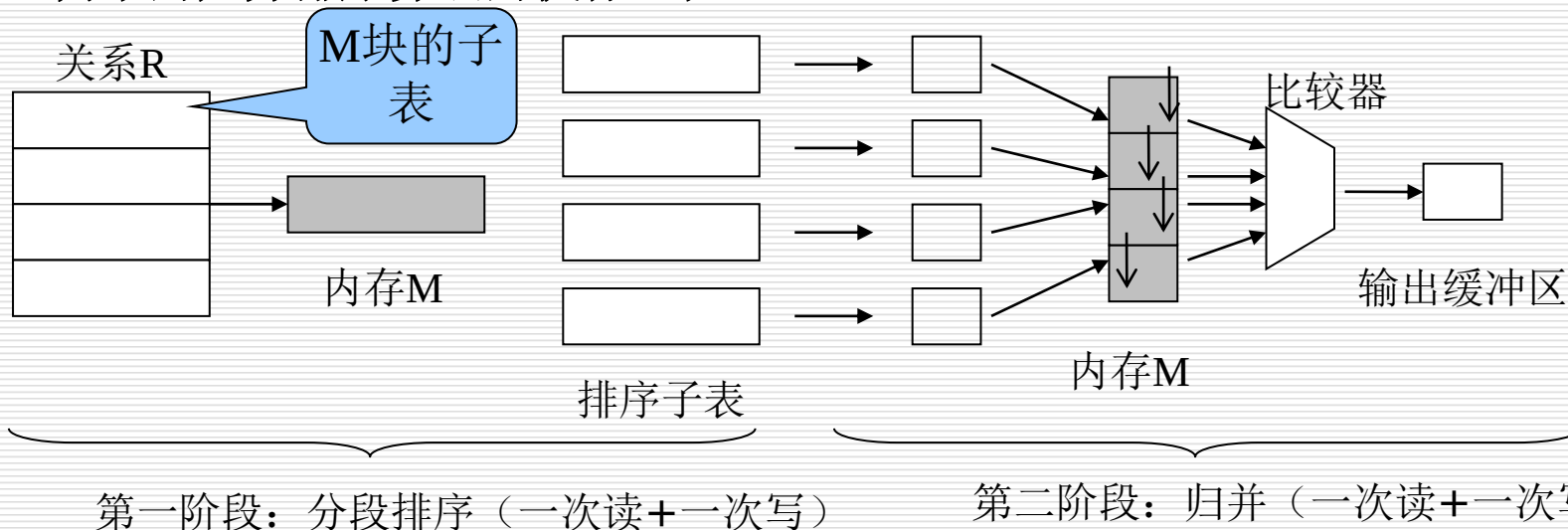
$500/(51-1)*2000+500=20500$ 次磁盘 IO。

枚举法优化技术

□ 基于排序的连接算法

- 首先将两个关系按照连接属性进行排序(分段排序+归并)
- 然后按照连接属性的顺序扫描两个关系，同时对两个关系中的元组执行连接操作。

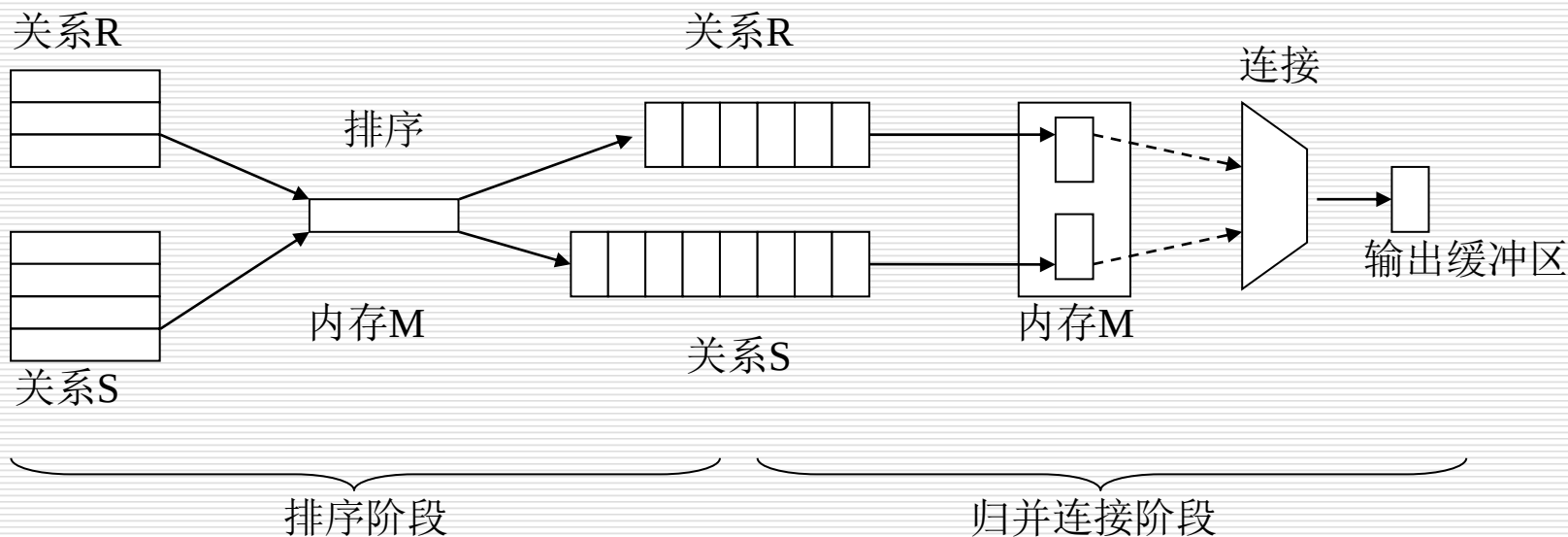
□ 简单的归并排序算法的执行过程



枚举法优化技术

简单基于排序的连接算法

- 对已经按照连接属性排序的两个关系，按照顺序读取关系中的块到内存中执行连接操作。



■ 执行代价: $C_{\text{join}} = C_{\text{排序I/O}} + C_{\text{连接I/O}} = C_{\text{分段排序I/O}} + C_{\text{归并I/O}} + C_{\text{连接I/O}}$

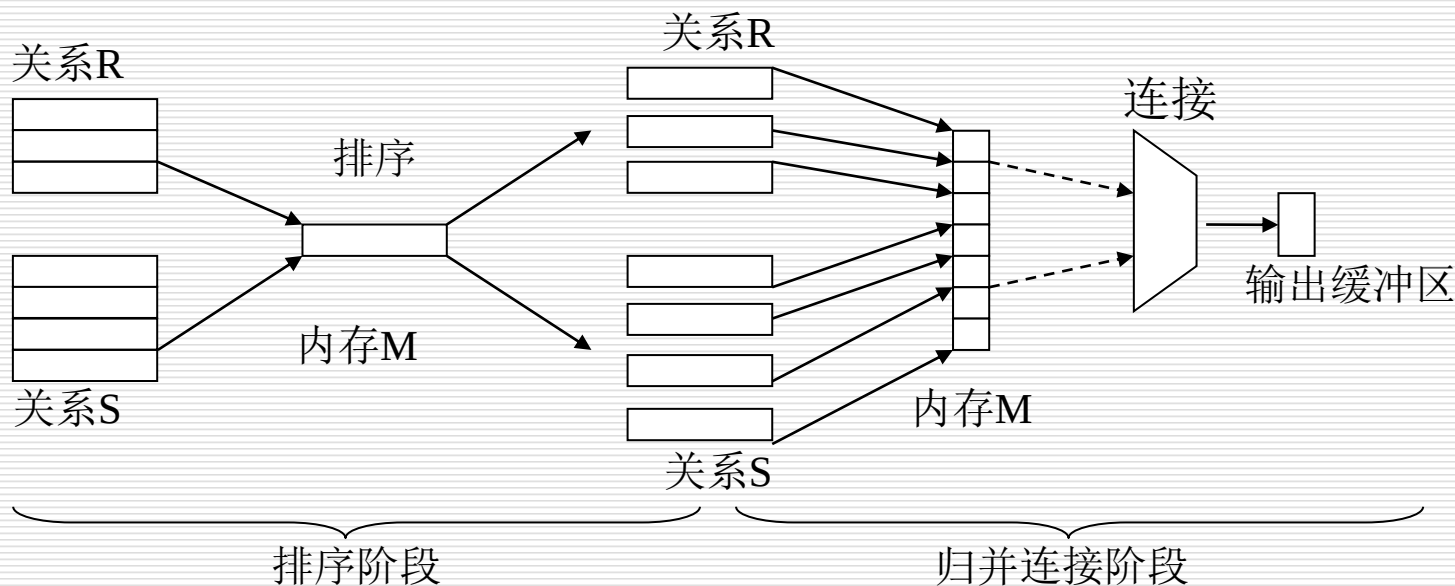
$$= 2(\text{Block}(R) + \text{Block}(S)) + 2(\text{Block}(R) + \text{Block}(S)) + (\text{Block}(R) + \text{Block}(S))$$

$$= 5(\text{Block}(R) + \text{Block}(S))$$

枚举法优化技术

□ 归并排序连接算法

- 其思想是将排序的第二阶段与归并连接阶段合并，这样可以节省一次对关系的读写操作。



- 执行代价为: $C_{\text{join}} = 3(\text{Block}(R) + \text{Block}(S))$

枚举法优化技术

□ 哈希连接算法

- 使用同一个哈希函数，对进行连接的两个关系**R**和**S**中的元组的连接属性值进行散列，具有相同属性值的元组会出现在相同**Hash**数值的桶中，然后对对应的桶中的元组执行连接
- 执行代价为： $C_{\text{join}} = 3(\text{Block}(R) + \text{Block}(S))$
 - ✓ 哈希代价：一次读写 $2(\text{Block}(R) + \text{Block}(S))$
 - ✓ 桶连接：若一个桶能全部装入内存缓冲区块中，需要一次读取操作 $(\text{Block}(R) + \text{Block}(S))$

枚举法优化技术

□ 连接关系的传输方法

当参与连接的两个关系在不同场地时，需要将它们传输到同一场地执行连接操作。

■ 全体传输

$$C_{\text{tran}} = [(\text{Card}(R) * \text{Length}(R)) / m] * C_{\text{mes}}$$

其中 m 为报文容量， C_{mes} 为报文的单位传输代价

■ 按需传输

$$C_{\text{tran}} = l * C_{\text{mes}} + [(\text{Card}(R') * \text{Length}(R')) / m] * C_{\text{mes}}$$

其中 l 为请求报文， R' 为需要传输的关系

集中式系统中的查询优化算法

分布式查询优化技术是在集中式查询优化技术基础上的扩展，其中增加了对通信代价的评估，主要优化方法：

- **INGRES** – dynamic optimization
- **System R** – static optimization based on exhaustive search

集中式系统中的查询优化算法

□ INGRES

INGRES系统使用的是一种动态查询优化算法，查询优化的过程划分为两个阶段：

第一阶段：基于演算代数的查询分解（decomposition）或称（*Detachment*）。即将一个查询分解为一个查询序列，序列中的每个查询包含一个独立的关系及在这个关系上的查询谓词。

第二阶段：查询优化（optimization）或查询替换（*substitution*）。针对序列中每个独立的查询，使用单变量查询处理器（one-variable query processor, OVQP）为每个独立查询中的逻辑操作选择合适的物理操作。

集中式系统中的查询优化算法

□ INGRES

举例：假设有如下更新操作“如果Jones是在1楼工作，则将其工资提高10%”，相应的QUEL语句如下：

```
RANGE OF E IS EMPLOYEE
RANGE OF D IS DEPT
REPLACE E.SALARY = 1.1 * E.SALARY
WHERE E.NAME = "Jones" AND
E.DEPT = D.DEPT AND D.FLOOR = 1
```

其中，**E**，**D**为变量

□ 优化思想

一个包含 n 个变量的查询 q 可以被分解为一个连续的查询序列

$q_1 \rightarrow q_2 \rightarrow \dots \rightarrow q_n$ ，其中 $q_i \rightarrow q_{i+1}$ 表示 q_i 先于 q_{i+1} 执行，并且 q_i 执行结果将被后续查询 q_{i+1} 所使用。

集中式系统中的查询优化算法

□ INGRES

- **单变量查询 (One-variable query)** : 包含 一个变量的查询。
- **多变量查询 (Multivariable query)** : 包含一个以上变量的查询。

QUEL(Query Language)查询可等价转换为**SQL**语句

□ 优化策略 (**General strategy of INGRES**)

- ✓ 将多变量查询转换为一系列单变量查询;
- ✓ 由单变量查询处理器**OVQP (One Variable Query Processor)**处理单变量查询, 并选择优化的处理方法, 例如 **index, sequential scan, etc.**
- 采用查询**拆分(Detachment)** 和**替换(substitution)**技术

集中式系统中的查询优化算法

□拆分(Detachment)

对一个给定的查询 q ，将其分解为 $q' \rightarrow q''$ ，其中 q'' 仅包含一个公共变量，该公共变量是 q' 的结果。

□元组替换(Tuple substitution)

使用实际的值取代每个元组的值，并简化查询

$$q(V_1, V_2, \dots, V_n) \rightarrow (q'(t_1, V_2, V_2, \dots, V_n), t_1 \in R)$$

集中式系统中的查询优化算法

□ 拆分(**Detachment**)-例如: Names of employees working on CAD/CAM project

q: SELECT E.ENAME
 FROM E, G, J
 WHERE E.ENO = G.ENO AND J.JNO = G.JNO
 AND **J.JNAME = "CAD/CAM"**

q 分解为 $q1 \rightarrow q21 \rightarrow q22$:

q1: SELECT J.JNO INTO **JVAR**
 FROM J
 WHERE **JNAME="CAD/CAM"**

q2 : SELECT E.ENAME
 FROM E, G, **JVAR**
 WHERE E.ENO = G.GNO AND G.JNO = **JVAR.JNO**

q2可通过拆分**G**进一步处理

集中式系统中的查询优化算法

□ 拆分 (Detachment)-举例

q21: SELECT G.ENO INTO GVAR
FROM G, JVAR
WHERE G.JNO=JVAR.JNO

q22: SELECT E.ENAME
FROM E, GVAR
WHERE E.ENO=GVAR.ENO

□ 元组替换 (Tuple substitution)

■ 对于一个 n 变量查询 q ，选择一个变量 (假设为关系 R_1)，用关系 R_1 的元组去替换查询 q ，得到包含 $(n-1)$ 个变量的关系。

表示: $q(V_1, V_2, \dots, V_n) \rightarrow (q'(t_1, V_2, V_2, \dots, V_n), t_1 \in R)$

集中式系统中的查询优化算法

□元组替换-举例

q: SELECT E.ENAME
 FROM E, G
 WHERE E.ENO = G.ENO
 $G=\{(E1,J1,...), (E2,J1,...), (E3,J2,...)\}$

用 **G** 关系的元组替换产生如下三个查询:

q1: SELECT E.ENAME
 FROM E
 WHERE E.ENO = "E1"

q2: SELECT E.ENAME
 FROM E
 WHERE E.ENO = "E2"

q3: SELECT E.ENAME
 FROM E
 WHERE E.ENO = "E3"

集中式系统中的查询优化算法

□ System R 中的查询优化算法

- System R中的查询优化算法是关系数据库使用较多的算法。

输入：一个SQL语言描述的查询被解析后的关系代数树

算法：根据该树描述的逻辑查询计划树，生成大量的物理执行计划，并最终从中选择出较优的一个物理执行计划。

- System R使用磁盘IO和CPU时间来评估物理执行计划代价。
- System R主要使用动态规划算法来选择代价最小的执行计划。

集中式系统中的查询优化算法

□ **System R**的查询优化算法中，主要考虑以下两个问题：

- 用于实现逻辑操作符的物理操作。可以分为在单独关系上的**选择谓词**操作和关系连接的**二元操作**。

选择代价最小执行方法，
如索引/全表扫描

嵌套循环和排序归并

- 相似操作的排序，主要考虑多关系之间的连接操作。（ **n** 个关系的连接操作将有 **$n!$** 个排列，因此不能对每个排列都进行代价估计。**System R**采用了**动态规划方法**）

集中式系统中的查询优化算法

□ System R-举例

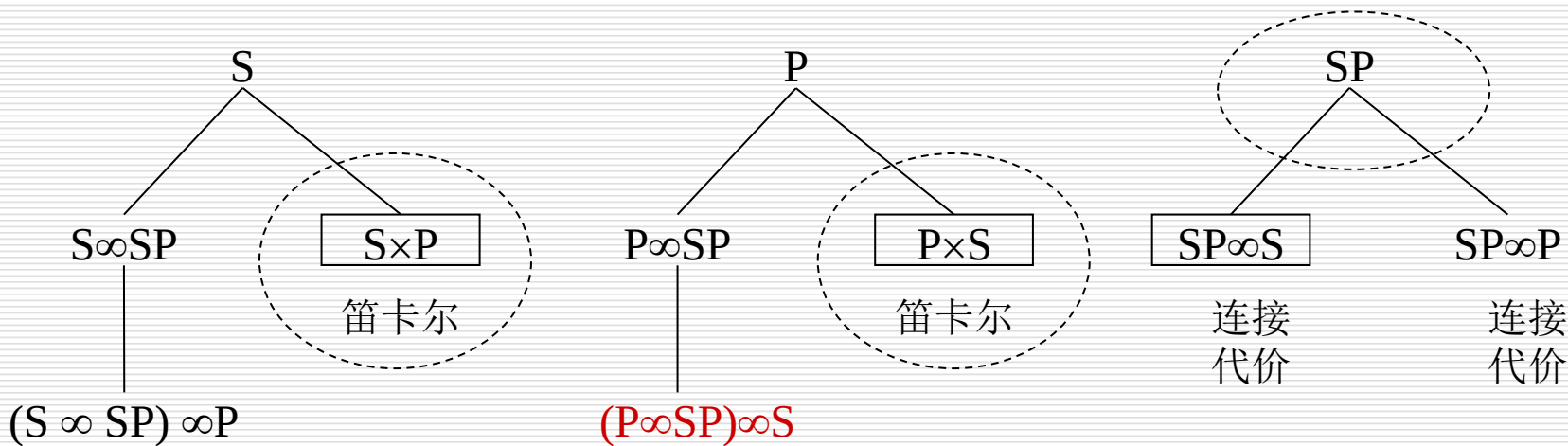
```
SELECT S.SNAME  
FROM SUPPLIER S, PARTS P, SUPPLY SP  
WHERE P.PNAME= 'Bolt' AND SP.S#=S.S#  
AND SP.P#=P.P#
```

假设PARTS的PNAME属性上有索引。

集中式系统中的查询优化算法

□ System R-举例

3个关系连接，6种查询的物理执行计划。



- ▶ $S \times P$ 和 $P \times S$ 是笛卡尔积运算，因此除去；
 - ▶ 较小的关系作外关系，执行代价较小，(假设 $SP \infty P > P \infty SP$)，则除去 $SP \infty P$ 。
- 同理，除去 $SP \infty S$ ；
- ▶ 由于在关系 P. PNAME 上有索引，因此选择 $(P \infty SP)$ ，除去 $SP \infty S$ 。
 - ▶ 当加入第三个关系后，得到连接序列 $(P \infty SP) \infty S$ 。

集中式系统中的查询优化算法

□ System R-举例

```
q1:  SELECT  E.ENAME  
      RFOM    E, G, J  
      WHERE  E.ENO = G.ENO AND J.JNO = G.JNO  
            AND J.JNAME = "CAD/CAM"
```

假设存在下列索引:

E has an index on ENO

G has an index on JNO

J has an index on JNO and an index on JNAME

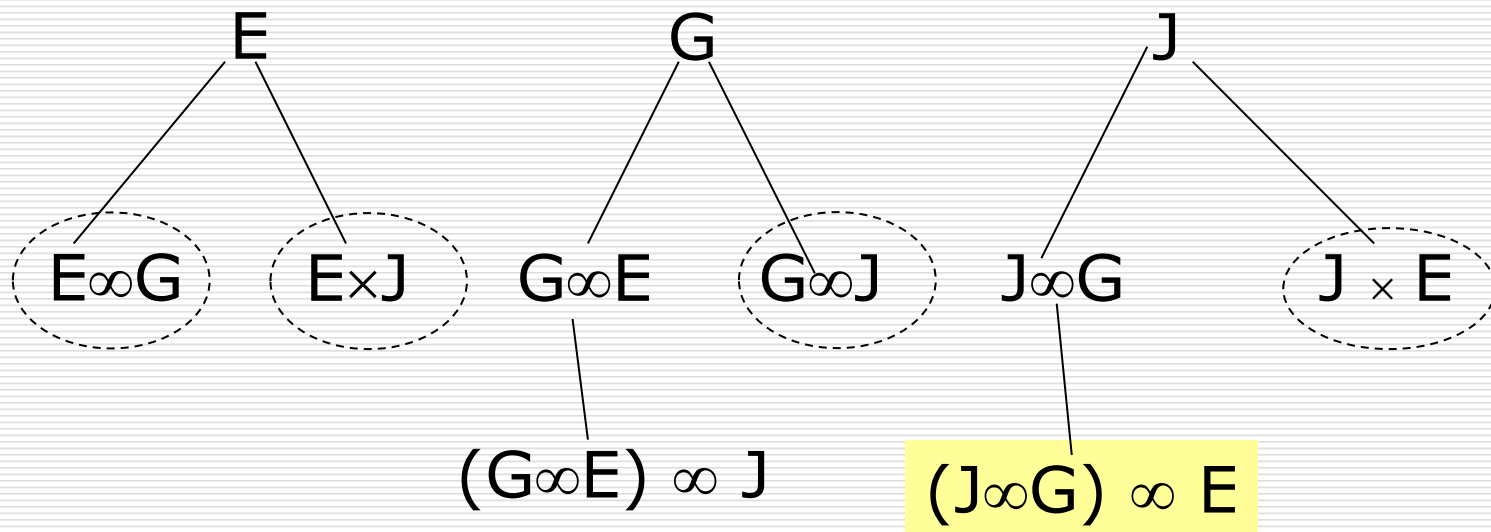
首先选择单关系的访问方法:

E: sequential scan (because there is no selection on E)

G: sequential scan (because there is no selection on G)

J: index on JNAME

集中式系统中的查询优化算法



假设 $E \infty G > G \infty E$, $G \infty J > J \infty G$

选择 $(J \infty G) \infty E$: 选择具有索引 $JNAME$ 的 J ; 再基于索引 JNO 与 G 实现 Join 操作; 再基于索引 ENO 与 E 实现 Join 操作。

分布式查询优化问题

- 代价模型
 - 多查询优化
 - 启发式规则裁剪
- 大的查询集合
 - 优化 **select-project-join** 查询
 - 也需要处理复杂查询 (如 **unions, disjunctions, aggregations** 和 **sorting**)
- 平衡优化代价与执行代价
 - 启发式规则裁剪 **heuristics to cut down on alternatives**
 - 控制搜索策略
- 优化与重优化间隔
 - 数据库概要信息变化时需要重优化

分布式查询优化算法

□ 分布式系统中的查询优化算法

■ Distributed INGRES方法

■ System R*方法

□ Distributed INGRES方法

当一个查询 Q 被分解为一组不可规约的子查询 $q_1, q_2, \dots, q_n$ 后,

- 集中式环境中可以简单地顺序执行每个子查询。
- 在分布式环境中, 由于关系以分片的形式存储在多个站点上, 因此子查询可能还需要继续细分以便能够在多个场地上执行, 而这将产生在站点之间复制关系所需的通信代价。
- 一种简单的优化方式是优先处理无前继查询且所涉及的关系较小的子查询。一方面可以获得较小的中间结果, 另一方面可以减少复制关系所需的传输代价。

分布式查询优化算法

□ Distributed INGRES方法

执行过程大致如下：

- 1，首先，所有在单关系上的**选择和投影操作**会被下推到数据所在场地执行；
- 2，使用INGRES中的**归约算法**，将包含 n 个关系变量的查询 q 分解为一个连续的查询序列 $q_1 \rightarrow q_2 \rightarrow \dots \rightarrow q_n$ ，由于单关系变量的查询已经处理结束（第1步），这里仅考虑不可归约的查询序列。
- 3，对于查询归约后获得的 n 个不可归约子查询，算法每次选择其中一个子查询进行处理。每次执行哪个子查询是根据查询结构、关系分片大小和关系分片的位置分布选择的，主要根据查询序列的**代价估计**来确定子查询的执行顺序（用一种贪婪算法）。
- 4，如果这个不可规约子查询能够在某个独立的站点执行而不需要复制关系分片，则直接跳转到第8步。否则需要复制关系分片。

分布式查询优化算法

□ Distributed INGRES方法

执行过程大致如下（续）：

- 5，选择查询的执行站点。根据**分片**涉及的站点，以及网络的拓扑结构（点到点或广播），对所有可能的执行场地进行考虑。
- 6，生成**查询执行策略**。假设有 n 个关系的子查询，一个执行查询的站点包含其中一个关系的一个分片，则必须将其他 $n-1$ 个关系全部复制到这个站点上才能够满足执行查询的条件。
- 7，根据第5步中的执行策略**复制**各关系的分片到指定的站点。为了对查询进行优化，仅传输与查询相关的分片，并可以使用广播形式提高传输效率。
- 8，由主INGRES站点向所有被选择的站点**广播查询**，查询被传输到各站点后，在各站点进行本地查询处理。如果所有站点上的本地查询都没有发生错误，则移除已执行的查询，必要时还要修改剩余关系变量的查询范围。
- 9，返回第3步继续执行，直到处理完全部查询序列。

分布式查询优化算法

□ Distributed INGRES方法-举例

核心：选择哪个子查询执行；查询执行策略的选择

■ 例如：存在三个关系：

SUPPLIER (SNO, SNAME, CITY)

PROJECT(JNO, JNAME, CITY)

SUPPLY(SNO, JNO, AMOUNT)

■ 存在场地：

S1: PROJECT (200元组), SUPPLIER1 (50元组)

S2: SUPPLY (400元组), SUPPLIER2 (50元组)

■ 要查询“供货商及其所供应的工程属于同一城市的供货商名称和工程名称”，查询的SQL语句如下：

SELECT S.SNAME, J.JNAME

FROM SUPPLIER S, PROJECT J, SUPPLY SJ

WHERE S.SNO=SJ.SNO and SJ.JNO=J.JNO and S.CITY=J.CITY



SELECT S.SNAME, J.JNAME

FROM SUPPLIER S, PROJECT J, SUPPLY SJ

WHERE S.SNO=SJ.SNO and SJ.JNO=J.JNO and S.CITY=J.CITY

分布式查询优化算法

□ Distributed INGRES方法-举例

■ 查询划分可有以下几种候选策略：

■ **Q1: SELECT S.SNAME, J.JNAME INTO temp**
FROM SUPPLIER S, PROJECT J, SUPPLY SJ
WHERE S.SNO=SJ.SNO and SJ.JNO=J.JNO and S.CITY=J.CITY

■ **Q2: SELECT S.SNAME, S.CITY, SJ.JNO INTO temp**
FROM SUPPLIER S, SUPPLY SJ WHERE S.SNO = SJ.SNO

■ **Q3: SELECT S.SNAME, S.SNO, J.JNAME, J.JNO INTO temp**
FROM SUPPLIER S, PROJECT J
WHERE S.CITY = J.CITY

■ **Q4: SELECT J.JNAME, J.CITY, SJ.SNO INTO temp**
FROM PROJECT J, SUPPLY SJ
WHERE J.JNO = SJ.JNO

Q1:S1上关系
PROJECT的200
个元组和
SUPPLIER1的
50个元组复制到
S2上执行连接。

S1: PROJECT (200元组), SUPPLIER1 (50元组)

S2: SUPPLY (400元组), SUPPLIER2 (50元组)



SELECT S.SNAME, J.JNAME

FROM SUPPLIER S, PROJECT J, SUPPLY SJ

WHERE S.SNO=SJ.SNO and SJ.JNO=J.JNO and S.CITY=J.CITY

分布式查询优化算法

□ Distributed INGRES方法-举例

■ Q2: **SELECT S.SNAME, S.CITY, SJ.JNO INTO temp**

FROM SUPPLIER S, SUPPLY SJ WHERE S.SNO = SJ.SNO

Q2: S1上的SUPPLIER1复制到S2，代价是50个元组。Q2的执行结果将被传输到S1执行剩余的查询： $J \bowtie temp$

SELECT T.SNAME, J.JNAME

FROM PROJECT J, temp T

WHERE T.CITY=J.CITY and T.JNO=SJ.JNO

■ 需要根据在关系和关系属性值上的相关特征参数进行评估执行代价。从中选择一个代价小的策略（参见教材P183）

S1: PROJECT (200元组), SUPPLIER1 (50元组)

S2: SUPPLY (400元组), SUPPLIER2 (50元组)

分布式查询优化算法

□ System R*方法

- **R***系统中的分布式查询优化算法是基于**System R**系统的查询优化器，因此其中同样使用**穷举法**将所有可能的查询执行策略进行代价估计，并选择其中**代价最小**的作为最终执行策略。
- **R***系统的实现中**不支持关系的分片和副本**，因此依然是对整个关系操作的优化。

□ 与集中式的**System R**相比，**R***系统的查询优化器还要考虑查询中连接操作的执行**场地**的选择和场地间数据传输的代价。

如：对于两个关系**R**和**S**的连接操作执行场地，有三种可选策略：

- 选择**R**所在的场地作为执行场地；
- 选择**S**所在的场地作为执行场地；
- 选择其它场地作为执行场地。

分布式查询优化算法

□ System R*方法

- 策略的选择要根据查询的情况和关系的相关统计信息。
- 目标：减少通信代价。

例如，

- ✓ 如果 $\text{Size}(R) > \text{Size}(S)$ ，则选择R所在场地作为执行场地，反之则选择S所在场地作为执行场地。
- ✓ 假设连接涉及三个关系，并有连接顺序 $(R \bowtie S) \bowtie T$ ，即先对R和S进行连接，执行结果再与T连接，此时如果有R和S关系均较小但连接的结果关系较大，则选择关系T所在的场地作为执行场地能够获得最小的通信代价。
- System R*系统主要采用两种数据传输方式：
 - ✓ 全体传输（Ship-whole）
 - ✓ 按需传输（fetch-as-needed）

分布式查询优化算法

□ System R*方法的四种连接执行策略

在代价模型中，用 C_{lp} 表示局部查询处理的IO和CPU代价， C_{com} 表示通信代价， t 表示关系 S 中与关系 R 的元组匹配的平均元组数量。 $C_{lp}(t)$ 表示从关系 S 中查询与 R 元组匹配的元组的处理代价。假设使用嵌套循环连接算法。 R 是外关系。

■ 全体传输外关系到内关系所在场地

$$C_{total} = C_{lp}(\text{Card}(R)) + C_{com}(\text{Size}(R)) + C_{lp}(t) * \text{Card}(R)$$

■ 全体传输内关系到外关系所在场地（内关系需要在外关系所在的场地的磁盘上保存）

$$C_{total} = C_{lp}(\text{Card}(S)) + C_{com}(\text{Size}(S)) + C_{lp}(\text{Card}(S)) + C_{lp}(\text{Card}(R)) + C_{lp}(t) * \text{Card}(R)$$

分布式查询优化算法

□ System R*方法的四种连接执行策略

■ 按需传输内关系元组

$$C_{\text{total}} = C_{\text{lp}}(\pi_A(R)) + C_{\text{com}}(\text{Length}(A)) * \text{Card}(R) + C_{\text{lp}}(t) * \text{Card}(R) \\ + C_{\text{com}}(t * \text{Card}(R) * \text{Length}(S))$$

■ 在其它场地执行连接

$$C_{\text{total}} = C_{\text{lp}}(\text{Card}(S)) + C_{\text{com}}(\text{Size}(S)) + C_{\text{lp}}(\text{Card}(S)) \\ + C_{\text{lp}}(\text{Card}(R)) + C_{\text{com}}(\text{Size}(R)) + C_{\text{lp}}(t) * \text{Card}(R)$$

大数据数据库的查询处理与优化

□ 键值二级索引 (Secondary Index for Key-Value Model)

■ 键值模型

□ 适于基于行键 (RowKey) 的单点查询和范围查询

■ 构建二级索引-支持类似于关系数据库中依属性列的查询

□ 列族模型数据二级索引：存储在采用键值模型的列族中

□ 二级索引类型：（1）宽行型二级索引；（2）超列型二级索引；（3）组合型二级索引。



大数据数据库的查询处理与优化

□ 键值二级索引

Column Family: EMP-WORK			
Key(ENO)	Columns		
001	Column Key	Column Value	Timestamp
	"ENAME"	"Bob"	4
	"SALARY"	"3500"	4
	"DNO"	"103"	4
002	Column Key	Column Value	Timestamp
	"ENAME"	"Lucy"	6
	"SALARY"	"2500"	6
	"DNO"	"102"	6
003	Column Key	Column Value	Timestamp
	"ENAME"	"John"	3
	"SALARY"	"2300"	3
	"DNO"	"102"	3
004	Column Key	Column Value	Timestamp
	"ENAME"	"Tom"	7
	"SALARY"	"2700"	7
	"DNO"	"101"	7
005	Column Key	Column Value	Timestamp
	"ENAME"	"Gray"	8
	"SALARY"	"2300"	8
	"DNO"	"103"	8

Column Family: DepartmentIndex			
Key(DNO)	Columns		
101	Column Key	Column Value	Timestamp
	"004"	null	7

Column Family: DepartmentIndex				
102	Key	Super Columns		
	“Dept”	Key	Columns	
“101”		Column Key	Column Value	Timestamp
		“004”	null	7
“102”		Column Key	Column Value	Timestamp
		“002”	null	6
		“003”		
103				

组合型二级索引

Column Family: DepartmentIndex			
Key(DNO)	Columns		
101_004	Column Key	Column Value	Timestamp
	"004"	null	7
102_002	Column Key	Column Value	Timestamp
	"002"	null	6
102_003	Column Key	Column Value	Timestamp
	"003"	null	3
103_001	Column Key	Column Value	Timestamp
	"001"	null	4
103_005	Column Key	Column Value	Timestamp
	"005"	null	8

宽行型二级索引

超列型二级索引

分布/集中

组合型二级索引

有序性&聚簇性

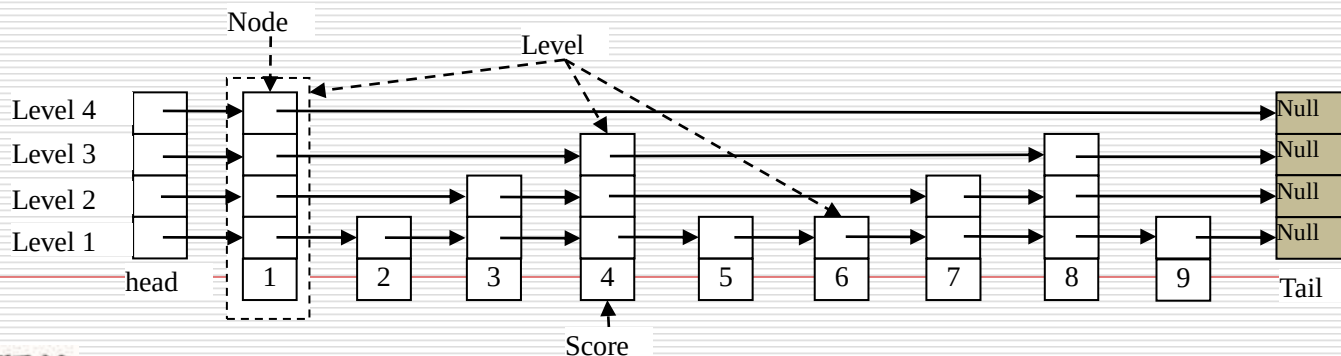
大数据数据库的查询处理与优化

□ 跳跃表 (Skiplist)

一种有序数据结构，通过在每个节点中维持多个指向其他节点的指针达到快速访问节点的目的。查找复杂度平均 $O(\log N)$ ，最坏 $O(N)$ 。

■ 基本结构

- 表头(head)-维护跳跃表的节点指针
- 节点(node)-保存索引元素值，每个节点有一层或多层
- 层(level)-保存指向该层下一个节点的指针
- 表尾(tail)-由null组成，是每层的最后一个指针



大数据数据库的查询处理与优化

□ 基于分析引擎的大数据库查询优化

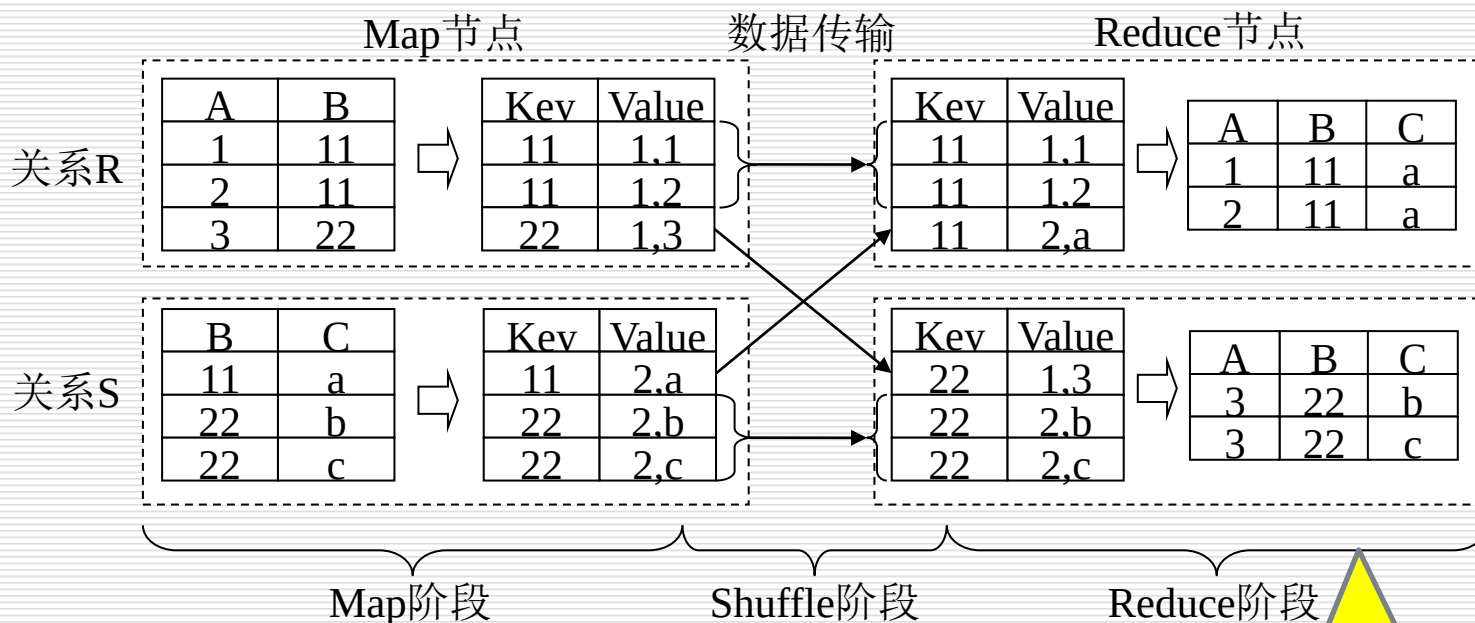
- 面向**事务型**应用的查询处理中，通常是针对某一指定键值（Key）的数据进行**简单查询**操作；
- 面向**分析型**应用的查询处理中，查询具有更高的复杂性，查询中不但包含各类聚合函数，还可能包含**复杂连接**操作；

借助**分布式或并行的分析引擎**来执行**分析型复杂操作**。

大数据数据库的查询处理与优化

□ 基于分析引擎的大数据库查询优化

■ 基于 **Hadoop** 的查询处理与优化



Reduce Side Join
算法样例

84

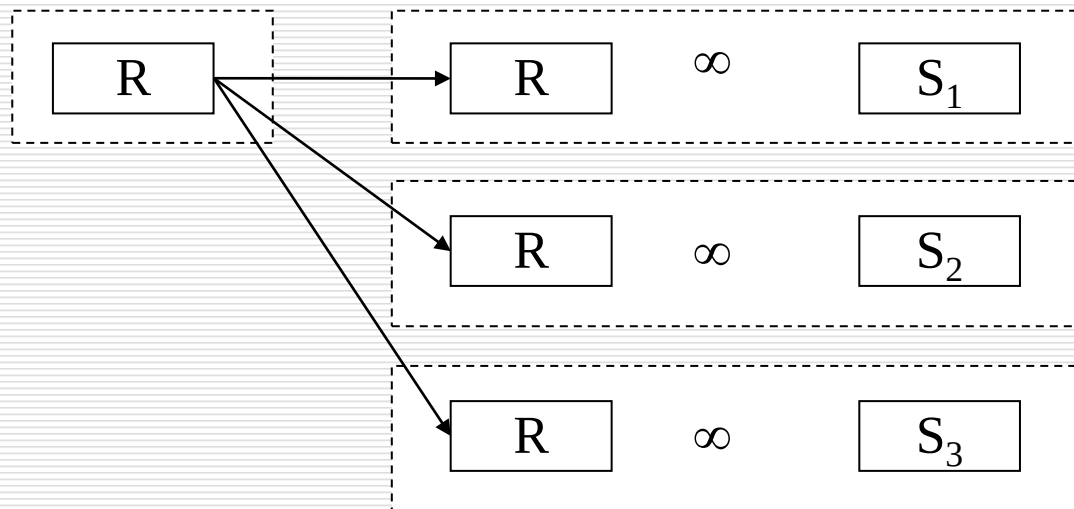
大数据数据库的查询处理与优化

□ 基于分析引擎的大数据库查询优化

■ 基于 **Hadoop** 的查询处理与优化

关系R存储节点

Map任务节点

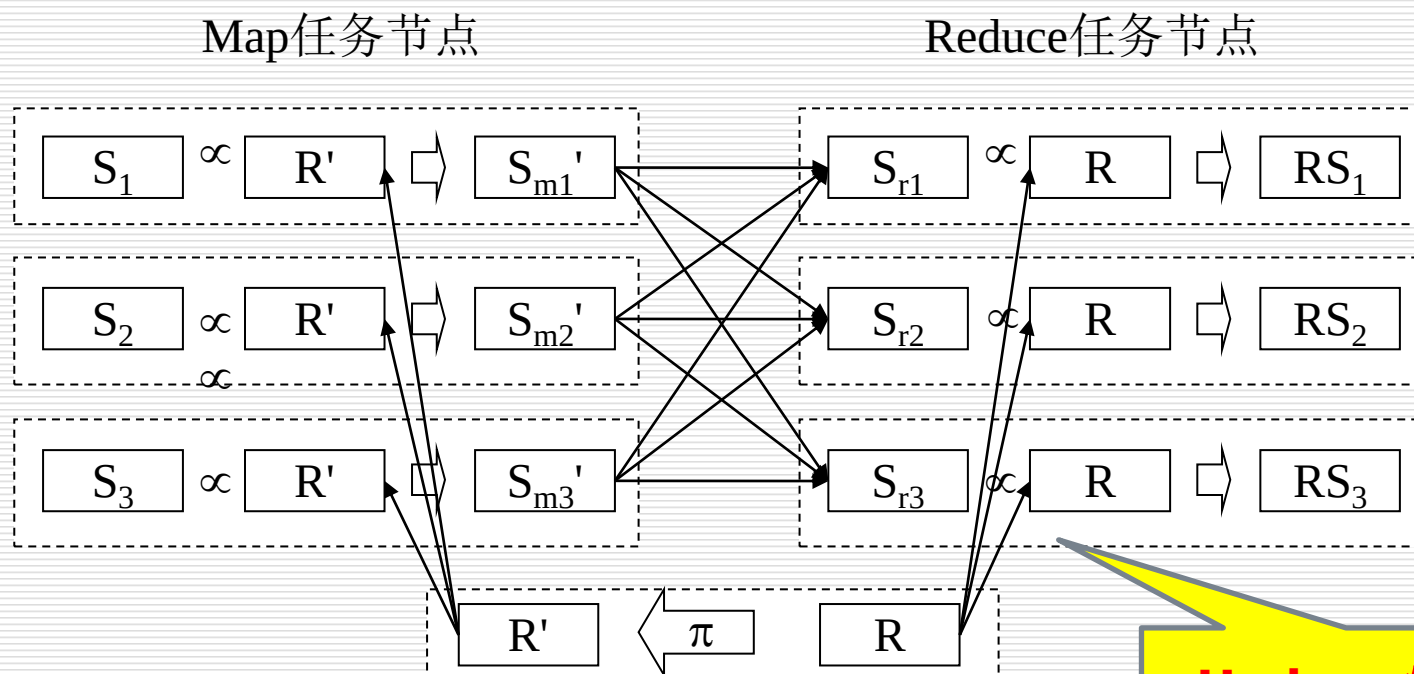


Map Side Join算法

大数据数据库的查询处理与优化

□ 基于分析引擎的大数据库查询优化

■ 基于Hadoop的查询处理与优化



**Hadoop的半连接
优化算法** 86



第五章 结束

